PANDAS COMPLETE HANDBOOK

Concepts · Questions · Answers · Code Examples

Using sales_dataset.csv — Real-world Practice

Alex | Data Analytics Portfolio | 2025

TABLE OF CONTENTS

Contents

- sort_values, sort_index
- rank
- nlargest, nsmallest

13. Statistical Methods

- corr, cov
- quantile, describe
- value_counts, mode

14. Advanced & Performance

- pipe, method chaining
- memory_usage
- categorical dtype

CHAPTER 01

Data Loading & Inspection

What is pandas?

pandas is a fast, powerful, flexible, and easy-to-use open-source data analysis and manipulation library built on top of NumPy. Every dataset in pandas is represented as a DataFrame (2D table) or Series (1D column).

1.1 Loading Data

Import pandas and load the CSV dataset:

```
import pandas as pd
df = pd.read_csv('sales_dataset.csv')
# Common parameters
df = pd.read_csv('sales_dataset.csv',
parse_dates=['order_date'],
index_col='order_id')
```

1.2 Inspecting a DataFrame

Property / Method	Returns	Example
df.shape	Tuple (rows, cols)	df.shape -> (1500, 25)
df.dtypes	Data type per column	df.dtypes
df.columns	Index of column names	df.columns.tolist()
df.index	Row index object	df.index
df.info()	Memory + dtypes summary	df.info()
df.describe()	Statistics for numeric cols	df.describe()
df.head(n)	First n rows (default 5)	df.head(10)
df.tail(n)	Last n rows	df.tail(3)
df.sample(n)	n random rows	df.sample(5, random_state=42)
df.nunique()	Unique values per col	df.nunique()
df.memory_usage()	Memory bytes per col	df.memory_usage(deep=True)

1.3 Questions & Answers

Q1:

Load sales_dataset.csv and display the first 5 rows. What shape does it have?

Answer:

Use pd.read_csv() then .head() and .shape.

```
df = pd.read_csv('sales_dataset.csv')
print(df.head())
print('Shape:', df.shape) # (1500, 25)
```

Q2:

How many unique products and customers exist in the dataset?

Answer:

Use .nunique() on specific columns.

```
print(df['product_id'].nunique()) # 15
print(df['customer_id'].nunique()) # 15
```

Q3:

List all column names and their data types.

Answer:

df.dtypes returns a Series mapping column -> dtype.

```
print(df.dtypes)
# or cleaner:
print(df.info())
```

Q4:

What is the memory usage of the entire DataFrame?

Answer:

df.memory_usage(deep=True) returns bytes per column; sum() gives total.

```
total = df.memory_usage(deep=True).sum()
print(f'Memory: {total / 1024:.1f} KB')
```

Q5:

How many rows have a non-null rating?

Answer:

Use .notna() or .count() which skips NaN by default.

```
print(df['rating'].count())
print(df['rating'].notna().sum())
```

CHAPTER 02

Indexing & Selection

loc vs iloc

loc selects by LABEL (row index name, column name). iloc selects by INTEGER POSITION (0-based). Use loc for named access, iloc for positional slicing.

2.1 Column Selection

```
# Single column -> Series
df['revenue']
# Multiple columns -> DataFrame
df[['product_name', 'revenue', 'profit']]
# All columns except some
df.drop(columns=['cogs', 'discount_amount'])
```

2.2 loc — Label-based

```
# Single row by label
df.loc['ORD-1000']
# Row slice + specific columns
df.loc['ORD-1000':'ORD-1010', ['product_name','revenue']]
# Boolean condition inside loc
df.loc[df['category'] == 'Electronics', ['product_name','unit_price']]
```

2.3 iloc — Position-based

```
# Row 0, all columns
df.iloc[0]
# Rows 0-4, columns 0-4
df.iloc[0:5, 0:5]
# Last row
df.iloc[-1]
# Every other row
df.iloc[::2]
```

2.4 at / iat — Single value access

```
# at: label-based single value (faster than loc for scalars)
df.at['ORD-1000', 'revenue']
# iat: position-based single value
df.iat[0, 14] # row 0, column index 14 (revenue)
```

2.5 Questions & Answers

Q1:

Select only the columns: product_name, category, revenue, profit.

Answer:

Pass a list of column names inside double brackets.

```
subset = df[['product_name','category','revenue','profit']]
print(subset.head())
```

Q2:

Using iloc, select rows 100 to 110 and the first 4 columns.

Answer:

iloc uses integer positions, both inclusive on start, exclusive on end.

```
print(df.iloc[100:111, 0:4])
```

Q3:

Using loc, get all rows where channel is 'Online' and return only salesperson and revenue.

Answer:

Combine boolean mask inside loc.

```
result = df.loc[df['channel']=='Online', ['salesperson','revenue']]
print(result.head())
```

Q4:

Get the revenue value of the very first and very last order using iat.

Answer:

iat[row, col] uses integer indices.

```
print('First:', df.iat[0, df.columns.get_loc('revenue')])
print('Last:', df.iat[-1, df.columns.get_loc('revenue')])
```

Q5:

Select every 100th row from the DataFrame.

Answer:

Use iloc with a step.

```
print(df.iloc[::100])
```

CHAPTER 03

Filtering & Boolean Operations

Boolean Masking

A boolean mask is a Series of True/False values. When you apply a condition (e.g. df['revenue'] > 500), pandas returns such a Series. Passing it back into df[mask] keeps only True rows. Combine with & (and), | (or), ~ (not).

3.1 Single Condition

```
# Revenue above 500
df[df['revenue'] > 500]
# Completed orders only
df[df['status'] == 'Completed']
# Electronics category
df[df['category'] == 'Electronics']
```

3.2 Multiple Conditions

```
# AND: Electronics AND revenue > 1000
df[(df['category'] == 'Electronics') & (df['revenue'] > 1000)]
# OR: Dhaka OR Chittagong
df[(df['region'] == 'Dhaka') | (df['region'] == 'Chittagong')]
# NOT: exclude Returned orders
df[~(df['status'] == 'Returned')]
```

Note: Always wrap each condition in parentheses when using & or |. Python operator precedence will cause bugs otherwise.

3.3 isin, between, query

```
# isin: multiple allowed values
df[df['region'].isin(['Dhaka', 'Sylhet', 'Chittagong'])]
# between: inclusive on both ends
df[df['revenue'].between(500, 2000)]
# query: SQL-like string syntax (convenient for interactive work)
df.query('category == "Electronics" and revenue > 1000')
df.query('discount_pct > 0 and status == "Completed"')
```

3.4 Questions & Answers

Q1:

Find all orders where profit is negative (loss-making orders).

Answer:

Filter df['profit'] < 0.

```
losses = df[df['profit'] < 0]
print(f'{len(losses)} loss-making orders')
print(losses[['order_id','product_name','revenue','profit']].head())
```

Q2:

Filter orders from Dhaka region where the customer type is Corporate.

Answer:

Combine two conditions with &.

```
result = df[(df['region']=='Dhaka') & (df['customer type']=='Corporate')]
print(result.shape)
```

Q3:

Use query() to find completed Electronics orders with a discount above 10%.

Answer:

query() accepts an expression string.

```
res = df.query('status == "Completed" and category == "Electronics"
and discount_pct > 10')
print(res[['product_name','discount_pct','revenue']])
```

Q4:

Find orders with revenue between 200 and 800 using between().

Answer:

between() is inclusive on both ends by default.

```
mid = df[df['revenue'].between(200, 800)]
print(mid.shape)
```

Q5:

List orders where the salesperson is either 'Arif Khan' or 'Sabrina Noor'.

Answer:

Use isin() for multiple values.

```
sales = df[df['salesperson'].isin(['Arif Khan','Sabrina Noor'])]
print(sales['salesperson'].value_counts())
```

CHAPTER 04

Data Cleaning & Missing Values

Missing Data in pandas

NaN (Not a Number) represents missing data. pandas uses NaN for floats and None/pd.NaT for objects/datetimes. Most functions skip NaN by default (skipna=True). Always check for missing data before analysis.

4.1 Detecting Missing Values

```
# Total missing per column
df.isnull().sum()
# Percentage missing
(df.isnull().sum() / len(df) * 100).round(2)
# Rows with any missing value
df[df.isnull().any(axis=1)]
# Heatmap view (requires seaborn)
import seaborn as sns, matplotlib.pyplot as plt
sns.heatmap(df.isnull(), cbar=False, yticklabels=False)
plt.show()
```

4.2 Filling Missing Values

```
# Fill rating with mean
df['rating'].fillna(df['rating'].mean(), inplace=True)
# Fill with forward fill (carry last known value)
df['rating'].fillna(method='ffill')
# Fill with a constant
df['return_reason'].fillna('None', inplace=True)
# Fill different columns with different values
df.fillna({'rating': df['rating'].median(),
'return_reason': 'N/A'})
```

4.3 Dropping Missing Values

```
# Drop rows where ANY column is NaN
df.dropna()
# Drop rows where ALL columns are NaN
df.dropna(how='all')
# Drop rows where rating is NaN only
df.dropna(subset=['rating'])
# Drop columns with > 50% missing
threshold = len(df) * 0.5
df.dropna(thresh=threshold, axis=1)
```

4.4 Duplicate Handling

```
# Check for duplicates
df.duplicated().sum()

# Show duplicate rows
df[df.duplicated(keep=False)]

# Drop duplicates keeping first occurrence
df.drop_duplicates(inplace=True)

# Duplicates on specific columns
df.drop_duplicates(subset=['order_id'], keep='first')
```

4.5 Data Type Conversion

```
# Convert order_date to datetime
df['order_date'] = pd.to_datetime(df['order_date'])

# Convert category to categorical (memory efficient)
df['category'] = df['category'].astype('category')
df['status'] = df['status'].astype('category')
df['channel'] = df['channel'].astype('category')

# Convert to numeric (coerce errors to NaN)
df['revenue'] = pd.to_numeric(df['revenue'], errors='coerce')
```

4.6 Questions & Answers

Q1:

How many missing values does the 'rating' column have? What percentage is that?

Answer:

Use isnull().sum() then divide by len(df).

```
missing = df['rating'].isnull().sum()
pct = missing / len(df) * 100
print(f'Missing: {missing} ({pct:.1f}%)')
```

Q2:

Fill missing ratings with the median rating rounded to 1 decimal place.

Answer:

Compute median first, then fillna.

```
median_rating = round(df['rating'].median(), 1)
df['rating'] = df['rating'].fillna(median_rating)
print(df['rating'].isnull().sum()) # should be 0
```

Q3:

Replace empty strings in return_reason with 'No Return'.

Answer:

Empty strings are not NaN. Replace with .replace().

```
df['return_reason'] = df['return_reason'].replace('', 'No Return')
print(df['return_reason'].value_counts())
```

Q4:

Convert the order_date column to datetime and verify.

Answer:

Use pd.to_datetime().

```
df['order_date'] = pd.to_datetime(df['order_date'])
print(df['order_date'].dtype) # datetime64[ns]
print(df['order_date'].head())
```

Q5:

Convert category, status, and channel to categorical dtype and check memory savings.

Answer:

Categorical dtype stores repeated strings as integers internally.

```
before = df.memory_usage(deep=True).sum()
for col in ['category','status','channel','payment_method']:
df[col] = df[col].astype('category')
after = df.memory_usage(deep=True).sum()
print(f'Saved {(before-after)/1024:.1f} KB')
```

CHAPTER 05

String Operations

The .str Accessor

The .str accessor gives you vectorized string methods on Series objects. Instead of looping row by row, you call df['col'].str.method() and pandas applies it to every element efficiently, respecting NaN values.

Method	Description	Example
str.upper()	Uppercase all	df['name'].str.upper()
str.lower()	Lowercase all	df['name'].str.lower()
str.strip()	Remove whitespace	df['col'].str.strip()
str.len()	Length of each string	df['col'].str.len()
str.contains(pat)	Boolean: contains pattern	df['col'].str.contains('Pro')
str.startswith(s)	Starts with string	df['col'].str.startswith('P')
str.endswith(s)	Ends with string	df['col'].str.endswith('s')
str.replace(a,b)	Replace substring	df['col'].str.replace('old','new')
str.split(sep)	Split into list	df['col'].str.split('-')
str.extract(pat)	Regex capture group	df['col'].str.extract(r'(\d+)')
str.get(i)	Get element from list	df['col'].str.split().str.get(0)
str.count(pat)	Count occurrences	df['col'].str.count('a')
str.slice(s,e)	Slice characters	df['col'].str.slice(0,3)
str.cat(sep=',')	Concatenate strings	df['col'].str.cat(sep=', ')

5.1 Common String Operations on Dataset

```
# Extract order number from order id (e.g. 'ORD-1000' -> 1000)
df['order_num'] = df['order_id'].str.extract(r'(\d+)').astype(int)

# Uppercase customer names
df['customer_upper'] = df['customer_name'].str.upper()

# Check if product name contains 'Pro'
df['is_pro'] = df['product_name'].str.contains('Pro', na=False)

# First word of product name
df['prod_first_word'] = df['product_name'].str.split().str.get(0)

# Count characters in product name
df['name_length'] = df['product_name'].str.len()
```

5.2 Questions & Answers

Q1:

Create a new column 'city_upper' with city names in uppercase.

Answer:

Use .str.upper() on the region column.

```
df['city_upper'] = df['region'].str.upper()
print(df['city_upper'].unique())
```

Q2:

Filter all products whose name starts with the word 'Laptop' or 'Monitor'.

Answer:

Combine str.startswith() with isin-like logic or str.contains.

```
mask = df['product_name'].str.contains('^(Laptop|Monitor)', regex=True)
print(df[mask]['product_name'].unique())
```

Q3:

Extract the numeric order number from order_id as an integer column.

Answer:

str.extract() with a capturing group, then cast to int.

```
df['order_num'] = df['order_id'].str.extract(r'(\d+)').astype(int)
print(df[['order_id','order_num']].head())
```

Q4:

Find all orders where the salesperson's name has more than 9 characters.

Answer:

Use str.len() to create a length Series and filter.

```
long_names = df[df['salesperson'].str.len() > 9]
print(long_names['salesperson'].unique())
```

Q5:

Replace 'In-Store' with 'Physical Store' in the channel column.

Answer:

Use str.replace() or .replace() on the Series.

```
df['channel'] = df['channel'].str.replace('In-Store','Physical Store',regex=False)
print(df['channel'].value_counts())
```

CHAPTER 06

Data Transformation & apply()

apply vs map vs applymap

apply() works on rows or columns (axis param). map() works element-wise on a Series and is best for value substitution. applymap() (now called map() in pandas 2.x) works element-wise on a DataFrame. Use vectorized ops when possible — apply() with a Python function is slower.

6.1 apply() on Series

```
# Classify profit as 'High', 'Medium', 'Low'
def classify_profit(p):
if p > 500: return 'High'
elif p > 100: return 'Medium'
else: return 'Low'
df['profit_tier'] = df['profit'].apply(classify_profit)
# Same with lambda
df['profit_tier'] = df['profit'].apply(
lambda p: 'High' if p>500 else ('Medium' if p>100 else 'Low'))
```

6.2 apply() on DataFrame (row-wise)

```
# Compute profit_margin % per row
df['margin_pct'] = df.apply(
lambda row: round(row['profit'] / row['revenue'] * 100, 2)
if row['revenue'] != 0 else 0,
axis=1 # axis=1 means apply across columns for each row
)
```

6.3 assign() — Chained Column Creation

```
# assign() returns a new DataFrame: use for method chaining
df2 = (df
.assign(margin_pct = lambda d: d['profit'] / d['revenue'] * 100)
.assign(is_high_value = lambda d: d['revenue'] > 1000)
.assign(year = lambda d: pd.to_datetime(d['order_date']).dt.year)
)
```

6.4 cut() and qcut() — Binning

```
# cut: define your own bin edges
df['revenue_band'] = pd.cut(
df['revenue'],
bins=[0, 100, 500, 1000, 5000],
labels=['Low','Medium','High','Premium']
```

```
)
# qcut: equal-frequency bins (quartiles)
df['revenue_quartile'] = pd.qcut(
df['revenue'], q=4, labels=['Q1','Q2','Q3','Q4']
)
```

6.5 get_dummies() — One-Hot Encoding

```
# One-hot encode 'category'
dummies = pd.get_dummies(df['category'], prefix='cat')
df = pd.concat([df, dummies], axis=1)
# drop_first=True removes one column to avoid multicollinearity
pd.get_dummies(df['channel'], drop_first=True)
```

6.6 Questions & Answers

Q1:
Create a column 'profit_margin' as profit/revenue * 100, rounded to 2 decimals.

Answer:
Use assign() or direct vectorized arithmetic.

```
df['profit_margin'] = (df['profit'] / df['revenue'] * 100).round(2)
print(df['profit_margin'].describe())
```

Q2:
Create a column 'order_size' that is 'Small' if qty<=5, 'Medium' if qty<=15, else 'Large'.

Answer:
Use apply() with a lambda or pd.cut().

```
df['order_size'] = df['quantity'].apply(
lambda q: 'Small' if q<=5 else ('Medium' if q<=15 else 'Large'))
print(df['order_size'].value_counts())
```

Q3:
Bin revenues into 4 equal-frequency quartiles using qcut.

Answer:
qcut() divides data into equal-count bins.

```
df['rev_q'] = pd.qcut(df['revenue'], q=4, labels=['Q1','Q2','Q3','Q4'])
print(df['rev_q'].value_counts())
```

Q4:
One-hot encode the 'payment_method' column.

Answer:
pd.get_dummies() creates binary indicator columns.

```
pay_dummies = pd.get_dummies(df['payment_method'], prefix='pay')
df = pd.concat([df, pay_dummies], axis=1)
print([c for c in df.columns if c.startswith('pay_')])
```

Q5:
Using assign(), chain three new columns: year, month_num, and is_weekend.

Answer:

pd.to_datetime gives .dt accessor for date parts.

```
df = (df.assign(
order_date = lambda d: pd.to_datetime(d['order_date']),
year = lambda d: d['order_date'].dt.year,
month_num = lambda d: d['order_date'].dt.month,
is_weekend = lambda d: d['order_date'].dt.dayofweek >= 5
))
print(df[['order_date','year','month_num','is_weekend']].head())
```

CHAPTER 07

Groupby & Aggregation

Split-Apply-Combine

groupby() implements the Split-Apply-Combine pattern: (1) Split the DataFrame into groups, (2) Apply a function to each group, (3) Combine results back. The result is indexed by the grouping keys.

7.1 Basic groupby

```
# Total revenue per category
df.groupby('category')['revenue'].sum()
# Multiple aggregations at once
df.groupby('category').agg(
total_revenue = ('revenue', 'sum'),
avg_profit = ('profit', 'mean'),
order_count = ('order_id','count'),
avg_rating = ('rating', 'mean')
).round(2)
```

7.2 Groupby Multiple Keys

```
# Revenue by category AND channel
df.groupby(['category','channel'])['revenue'].sum().unstack(fill_value=0)
# Salesperson performance by region
df.groupby(['region','salesperson']).agg(
orders = ('order_id', 'count'),
revenue = ('revenue', 'sum'),
profit = ('profit', 'sum')
).reset_index()
```

7.3 transform() — Group-level Values in Original Shape

```
# Add a column: each order's share of its category's total revenue
df['cat_total_rev'] = df.groupby('category')['revenue'].transform('sum')
df['rev_share'] = df['revenue'] / df['cat_total_rev'] * 100
# Rank within group
df['rank_in_cat'] = df.groupby('category')['revenue'].rank(ascending=False)
```

7.4 filter() — Keep Groups Meeting a Condition

```
# Keep only categories with more than 300 orders
big_cats = df.groupby('category').filter(lambda g: len(g) > 300)
# Keep only salespersons with avg rating >= 4.0
top_reps = df.groupby('salesperson').filter(
```

```
    lambda g: g['rating'].mean() >= 4.0
)
```

7.5 Questions & Answers

Q1:
Find total revenue, total profit, and order count per region.

Answer:
groupby region and aggregate three columns.

```
result = df.groupby('region').agg(
revenue = ('revenue','sum'),
profit = ('profit','sum'),
orders = ('order_id','count')
).sort_values('revenue', ascending=False)
print(result)
```

Q2:
Who is the top salesperson by total revenue in each region?

Answer:
groupby region+salesperson, sum revenue, then idxmax per group.

```
sp = df.groupby(['region','salesperson'])['revenue'].sum().reset_index()
top = sp.loc[sp.groupby('region')['revenue'].idxmax()]
print(top)
```

Q3:
Add a column showing average revenue for each product category (using transform).

Answer:
transform returns same-shape result aligned with the original DataFrame.

```
df['cat_avg_rev'] = df.groupby('category')['revenue'].transform('mean')
print(df[['product_name','category','revenue','cat_avg_rev']].head(10))
```

Q4:
Which payment methods generated above-average profit per order?

Answer:
groupby payment_method, compute mean profit, then filter.

```
pm = df.groupby('payment_method')['profit'].mean()
print(pm[pm > pm.mean()].sort_values(ascending=False))
```

Q5:
Compute the month-over-month revenue per category (groupby month + category).

Answer:
groupby month and category, sum revenue.

```
monthly = df.groupby(['month','category'])['revenue'].sum().reset_index()
monthly = monthly.sort_values(['category','month'])
monthly['mom_change'] = monthly.groupby('category')['revenue'].pct_change()*100
print(monthly.tail(12))
```

CHAPTER 08

Pivot Tables & Crosstabs

pivot_table vs pivot

pivot_table is the flexible, aggregating version (like Excel pivot tables). pivot() is for reshaping when values are already unique per (index, column) pair. crosstab is a shortcut for frequency counting between two categorical columns.

8.1 pivot_table

```
# Revenue by category (rows) and channel (columns)
pt = pd.pivot_table(
df,
values = 'revenue',
index = 'category',
columns = 'channel',
aggfunc = 'sum',
fill_value = 0,
margins = True, # adds row/column totals
margins_name = 'Total'
)
print(pt.round(0))
```

8.2 Multiple Values & Functions

```
pd.pivot_table(
df,
values = ['revenue', 'profit'],
index = ['region', 'category'],
columns = 'customer_type',
aggfunc = {'revenue': 'sum', 'profit': 'mean'}
)
```

8.3 crosstab

```
# Count of orders: channel vs status
pd.crosstab(df['channel'], df['status'])
# Normalize to percentages
pd.crosstab(df['channel'], df['status'], normalize='index').round(3)*100
# With values and aggfunc
pd.crosstab(
df['category'], df['customer_type'],
values=df['revenue'], aggfunc='sum'
)
```

8.4 stack & unstack

```
# unstack moves the last row-level index to columns
grouped = df.groupby(['category','channel'])['revenue'].sum()
wide = grouped.unstack(fill_value=0) # channel becomes columns
# stack reverses: move columns back to row index
long = wide.stack() # back to multi-index Series
```

8.5 Questions & Answers

Q1:
Create a pivot table showing average profit by region (rows) and customer_type (columns).

Answer:
Use aggfunc='mean'.

```
pt = pd.pivot_table(df, values='profit', index='region',
columns='customer_type', aggfunc='mean', fill_value=0)
print(pt.round(2))
```

Q2:
Use crosstab to count orders by payment_method and status.

Answer:
pd.crosstab takes two Series.

```
ct = pd.crosstab(df['payment_method'], df['status'])
print(ct)
```

Q3:
Show percentage of orders in each status per channel using normalize in crosstab.

Answer:
normalize='index' divides each row by its row total.

```
pct = pd.crosstab(df['channel'], df['status'], normalize='index')*100
print(pct.round(1))
```

Q4:
Create a pivot showing total revenue and total orders per quarter per category.

Answer:
Use multiple aggfunc values.

```
pt = pd.pivot_table(df, values=['revenue','order_id'],
index='quarter', columns='category',
aggfunc={'revenue':'sum','order_id':'count'})
print(pt)
```

Q5:
Reshape the category x channel revenue pivot using stack() to long format.

Answer:
stack() turns columns back into a row-level index.

```
pt = pd.pivot_table(df, values='revenue', index='category',
columns='channel', aggfunc='sum', fill_value=0)
long = pt.stack().reset_index(name='revenue')
print(long.head(10))
```

CHAPTER 09

Merging, Joining & Concatenation

Types of Joins

inner: only matching keys in both. left: all from left, NaN if no match on right. right: all from right. outer: all rows from both sides. Use on= for key column(s). suffixes= handles duplicate column names after merge.

9.1 pd.merge()

```
# Create a product reference table
product_ref = df[['product_id','product_name','category']].drop_duplicates()
# Create a customer reference table
customer_ref = df[['customer_id','customer_name','region','customer_type']].drop_duplicates()
# Inner merge (keep only matching rows)
merged = pd.merge(df, product_ref, on='product_id', how='inner',
suffixes=('','_ref'))
# Left join: keep all orders, add customer info
merged = pd.merge(df, customer_ref, on='customer_id', how='left')
```

9.2 pd.concat()

```
# Stack DataFrames vertically (same columns)
df_2023 = df[df['order_date'].str.startswith('2023')]
df_2024 = df[df['order_date'].str.startswith('2024')]
combined = pd.concat([df_2023, df_2024], ignore_index=True)
# Side by side (axis=1)
summary = pd.concat([
df.groupby('category')['revenue'].sum().rename('total_rev'),
df.groupby('category')['profit'].sum().rename('total_profit')
], axis=1)
```

9.3 DataFrame.join()

```
# join uses the index by default
rev = df.groupby('category')['revenue'].sum()
profit = df.groupby('category')['profit'].sum()
combined = rev.to_frame().join(profit.to_frame())
```

9.4 Questions & Answers

Q1:

Create a product summary table (product_id, avg revenue, total orders) and merge it back with a product reference.

Answer:

Build summary with groupby, then merge on product_id.

```
prod_sum = df.groupby('product_id').agg(
avg_rev=('revenue','mean'), orders=('order_id','count')
).reset_index()
prod_ref = df[['product_id','product_name','category']].drop_duplicates()
result = pd.merge(prod_ref, prod_sum, on='product_id')
print(result.sort_values('avg_rev', ascending=False))
```

Q2:

Split the dataset into 2023 and 2024 DataFrames then concat them back.

Answer:

Use string slicing or dt.year after parsing dates.

```
df['order_date'] = pd.to_datetime(df['order_date'])
df_23 = df[df['order_date'].dt.year == 2023]
df_24 = df[df['order_date'].dt.year == 2024]
full = pd.concat([df_23, df_24], ignore_index=True)
print(full.shape, df.shape)
```

Q3:

Create a region-level revenue summary and join it with a region-level order count.

Answer:

Build two Series indexed by region, then join.

```
rev = df.groupby('region')['revenue'].sum()
cnt = df.groupby('region')['order_id'].count()
summary = rev.to_frame('revenue').join(cnt.to_frame('orders'))
summary['avg_order_value'] = (summary['revenue'] / summary['orders']).round(2)
print(summary)
```

Q4:

Merge the orders with a discount lookup: for orders with 0 discount, label 'No Discount', else 'Discounted'.

Answer:

Build the lookup, then merge on discount_pct.

```
lookup = pd.DataFrame({'discount_pct':[0,5,10,15,20],
'discount_label':['No Discount','Low','Medium','High','Premium']})
df2 = pd.merge(df, lookup, on='discount_pct', how='left')
print(df2['discount_label'].value_counts())
```

CHAPTER 10

Time Series Analysis

datetime in pandas

pandas stores dates as datetime64[ns]. Always parse with pd.to_datetime(). The .dt accessor gives year, month, day, hour, dayofweek, quarter, and more. resample() is like groupby() but for time periods.

10.1 The .dt Accessor

```
df['order_date'] = pd.to_datetime(df['order_date'])
df['year'] = df['order_date'].dt.year
df['month'] = df['order_date'].dt.month
df['day'] = df['order_date'].dt.day
df['dayofweek'] = df['order_date'].dt.dayofweek # 0=Mon
df['day_name'] = df['order_date'].dt.day_name()
df['quarter'] = df['order_date'].dt.quarter
df['week'] = df['order_date'].dt.isocalendar().week
df['is_month_end']= df['order_date'].dt.is_month_end
```

10.2 resample()

```
# Set date as index first
ts = df.set_index('order_date').sort_index()
# Monthly revenue
monthly_rev = ts['revenue'].resample('ME').sum()
# Weekly order count
weekly_cnt = ts['order_id'].resample('W').count()
# Quarterly stats
quarterly = ts[['revenue','profit']].resample('QE').agg(
{'revenue':'sum', 'profit':'mean'}
)
```

10.3 shift() and diff()

```
# Lag: previous month's revenue
monthly_rev['prev_month'] = monthly_rev.shift(1)
# Month-over-month absolute change
monthly_rev['mom_change'] = monthly_rev['revenue'].diff()
# % change
monthly_rev['mom_pct'] = monthly_rev['revenue'].pct_change() * 100
```

10.4 Questions & Answers

Q1:

Extract year, month name, and day of week from order_date.

Answer:

Use .dt accessor after converting to datetime.

```
df['order_date'] = pd.to_datetime(df['order_date'])
df['year'] = df['order_date'].dt.year
df['month_name']= df['order_date'].dt.month_name()
df['day_name'] = df['order_date'].dt.day_name()
print(df[['order_date','year','month_name','day_name']].head())
```

Q2:

Find which day of the week has the highest average revenue.

Answer:

Group by day name, compute mean revenue.

```
df['day_name'] = pd.to_datetime(df['order_date']).dt.day_name()
daily = df.groupby('day_name')['revenue'].mean().sort_values(ascending=False)
print(daily)
```

Q3:

Compute monthly total revenue and show month-over-month % change.

Answer:

resample('ME') then pct_change().

```
ts = df.set_index(pd.to_datetime(df['order_date'])).sort_index()
monthly = ts['revenue'].resample('ME').sum()
monthly_df = monthly.to_frame()
monthly_df['pct_change'] = monthly_df['revenue'].pct_change()*100
print(monthly_df.round(1))
```

Q4:

Which quarter had the highest total profit across both years?

Answer:

Use resample('QE') or groupby quarter column.

```
ts = df.set_index(pd.to_datetime(df['order_date'])).sort_index()
qprofit = ts['profit'].resample('QE').sum()
print('Best quarter:', qprofit.idxmax(), '->', qprofit.max())
```

Q5:

Find all orders placed on weekends.

Answer:

dayofweek >= 5 means Saturday(5) or Sunday(6).

```
df['order_date'] = pd.to_datetime(df['order_date'])
weekends = df[df['order_date'].dt.dayofweek >= 5]
print(f'{len(weekends)} weekend orders')
```

CHAPTER 11

Rolling, Expanding & Window Functions

Window Functions

rolling(n) computes over the last n rows (fixed window). expanding() grows the window from the start to the current row (cumulative). ewm() gives more weight to recent observations. These are essential for smoothing time-series data.

11.1 rolling()

```
ts = df.set_index(pd.to_datetime(df['order_date'])).sort_index()
daily = ts['revenue'].resample('D').sum().fillna(0)
# 7-day rolling average (smooths daily noise)
daily_ma7 = daily.rolling(window=7).mean()
# 30-day rolling sum
daily_sum30 = daily.rolling(window=30).sum()
# Rolling standard deviation (volatility)
daily_std7 = daily.rolling(7).std()
```

11.2 expanding() and cumulative

```
# Cumulative revenue over time
daily['cumulative_rev'] = daily.expanding().sum()
# Running average
daily['running_avg'] = daily.expanding().mean()
# Direct cumulative functions
df['cumrev'] = df.sort_values('order_date')['revenue'].cumsum()
df['cummax'] = df.sort_values('order_date')['revenue'].cummax()
df['cumcount']= df.sort_values('order_date').groupby('category').cumcount()
```

11.3 ewm() — Exponential Weighted Mean

```
# More weight to recent data (alpha = smoothing factor)
daily['ema_7'] = daily.ewm(span=7, adjust=False).mean()
daily['ema_30']= daily.ewm(span=30, adjust=False).mean()
```

11.4 Questions & Answers

Q1:
Compute the 7-day rolling average of daily revenue.

Answer:
resample to daily, then rolling(7).mean().

```
ts = df.set_index(pd.to_datetime(df['order_date'])).sort_index()
daily = ts['revenue'].resample('D').sum().fillna(0)
```

```
daily_ma = daily.rolling(7).mean()
print(daily_ma.dropna().head(10))
```

Q2:

Compute the cumulative revenue over the full dataset sorted by date.

Answer:

Sort by order_date then cumsum().

```
df_sorted = df.sort_values('order_date')
df_sorted['cumrev'] = df_sorted['revenue'].cumsum()
print(df_sorted[['order_date','revenue','cumrev']].tail(10))
```

Q3:

Find the 30-day rolling maximum revenue to identify peak periods.

Answer:

rolling(30).max() on the daily revenue Series.

```
ts = df.set_index(pd.to_datetime(df['order_date'])).sort_index()
daily = ts['revenue'].resample('D').sum().fillna(0)
print(daily.rolling(30).max().dropna().describe())
```

Q4:

Calculate a 3-month expanding mean of monthly revenue.

Answer:

After resampling monthly, use expanding().mean().

```
ts = df.set_index(pd.to_datetime(df['order_date'])).sort_index()
monthly = ts['revenue'].resample('ME').sum()
monthly_exp = monthly.expanding().mean()
print(monthly_exp.round(2))
```

CHAPTER 12

Sorting & Ranking

12.1 sort_values()

```
# Sort by revenue descending
df.sort_values('revenue', ascending=False)
# Multi-column sort
df.sort_values(['category','revenue'], ascending=[True, False])
# Sort and reset index
df.sort_values('profit', ascending=False).reset_index(drop=True)
```

12.2 rank()

```
# Rank revenue (1 = highest), method handles ties
df['rev_rank'] = df['revenue'].rank(ascending=False, method='dense')
# Rank within group (rank per category)
df['rank_in_cat'] = df.groupby('category')['revenue'].rank(
ascending=False, method='min'
)
# rank() methods: 'average','min','max','first','dense'
```

12.3 nlargest / nsmallest

```
# Top 10 orders by revenue
df.nlargest(10, 'revenue')[['order_id','product_name','revenue']]
# 5 cheapest unit prices
df.nsmallest(5, 'unit_price')[['product_name','unit_price']]
# Top 3 salespersons by total revenue
df.groupby('salesperson')['revenue'].sum().nlargest(3)
```

12.4 Questions & Answers

Q1:
Show the top 10 orders by revenue with their product name, region, and salesperson.

Answer:
nlargest(10, 'revenue') then select columns.

```
top10 = df.nlargest(10,'revenue')[['order_id','product_name','region','salesperson','revenue']]
print(top10)
```

Q2:
Rank all products by their total revenue and show top 5.

Answer:
groupby product_name, sum revenue, rank, sort.

```
prod_rev = df.groupby('product_name')['revenue'].sum().reset_index()
prod_rev['rank'] = prod_rev['revenue'].rank(ascending=False, method='dense').astype(int)
print(prod_rev.nsmallest(5,'rank'))
```

Q3:

Add a column 'percentile' showing each order's revenue percentile (0-100).

Answer:

rank pct=True gives a 0-1 percentile, multiply by 100.

```
df['percentile'] = df['revenue'].rank(pct=True) * 100
print(df[['revenue','percentile']].describe())
```

Q4:

Sort by category A-Z, then within each category by profit descending.

Answer:

sort_values accepts a list of columns and matching ascending list.

```
sorted_df = df.sort_values(['category','profit'], ascending=[True, False])
print(sorted_df[['category','product_name','profit']].head(15))
```

CHAPTER 13

Statistical Methods

13.1 Descriptive Statistics

```
# Full summary
df.describe(include='all') # include='all' adds object columns

# Individual stats
df['revenue'].mean()
df['revenue'].median()
df['revenue'].std()
df['revenue'].var()
df['revenue'].skew() # skewness
df['revenue'].kurt() # kurtosis
df['revenue'].quantile([.25,.5,.75,.9,.99])
```

13.2 Correlation & Covariance

```
# Correlation matrix (numeric columns only)
df[['revenue','profit','unit price','quantity','rating']].corr().round(3)

# Specific correlation
df['revenue'].corr(df['profit']) # Pearson by default
df['revenue'].corr(df['profit'], method='spearman')

# Covariance
df[['revenue','profit']].cov()
```

13.3 value_counts & mode

```
# Most common category
df['category'].value_counts()
df['category'].value_counts(normalize=True).round(3) # proportions

# Mode (most frequent value)
df['payment method'].mode()[0]

# value_counts on numeric (with bins)
df['revenue'].value_counts(bins=5, sort=False)
```

13.4 Questions & Answers

Q1:
What is the mean, median, and standard deviation of revenue?

Answer:
Use the individual stat methods.

```
print(f'Mean: {df["revenue"].mean():.2f}')
print(f'Median: {df["revenue"].median():.2f}')
print(f'StdDev: {df["revenue"].std():.2f}')
```

Q2:

Compute the correlation between revenue and profit. Interpret the value.

Answer:

Pearson r: 1=perfect positive, 0=no linear, -1=perfect negative.

```
r = df['revenue'].corr(df['profit'])
print(f'Correlation: {r:.4f}')
# r close to 1 means strong positive relationship
```

Q3:

Find the 90th percentile of revenue (the top 10% threshold).

Answer:

Use .quantile(0.90).

```
p90 = df['revenue'].quantile(0.90)
print(f'90th percentile: {p90:.2f}')
print(f'Orders above p90: {(df["revenue"] > p90).sum()}')
```

Q4:

Which channel appears most often in the dataset?

Answer:

value_counts()[0] or mode()[0].

```
print(df['channel'].value_counts())
print('Most common:', df['channel'].mode()[0])
```

Q5:

Is the revenue distribution skewed? Check skewness and interpret.

Answer:

skew > 0 means right-skewed (long tail on right), < 0 means left-skewed.

```
sk = df['revenue'].skew()
print(f'Skewness: {sk:.3f}')
# positive = right skew = most orders are low-revenue, a few very large ones
```

CHAPTER 14

Advanced Operations & Performance

14.1 Method Chaining with pipe()

```
def add_margin(df):
df['margin'] = (df['profit'] / df['revenue'] * 100).round(2)
return df
def flag_top(df, n=100):
df['is_top'] = df['revenue'].rank(ascending=False) <= n
return df
# Readable pipeline
result = (
df
.query('status == "Completed"')
.assign(order_date = lambda d: pd.to_datetime(d['order_date']))
.pipe(add_margin)
.pipe(flag_top, n=50)
.sort_values('revenue', ascending=False)
.reset_index(drop=True)
)
```

14.2 Memory Optimization

```
# Check memory before
before = df.memory_usage(deep=True).sum() / 1024
print(f'Before: {before:.1f} KB')
# Downcast numerics
df['quantity'] = pd.to_numeric(df['quantity'], downcast='integer')
df['unit_price'] = pd.to_numeric(df['unit_price'], downcast='float')
df['revenue'] = pd.to_numeric(df['revenue'], downcast='float')
# Categoricals for low-cardinality string columns
for col in ['category','status','channel','region','payment_method']:
df[col] = df[col].astype('category')
after = df.memory_usage(deep=True).sum() / 1024
print(f'After: {after:.1f} KB (saved {before-after:.1f} KB)')
```

14.3 Useful Utility Methods

```
# Rename columns
df.rename(columns={'customer_name':'client','order_date':'date'}, inplace=True)
# Reorder columns
cols = ['order_id','order_date','product_name','category','revenue','profit']
df_reordered = df[cols + [c for c in df.columns if c not in cols]]
# Melt: wide to long
melted = df.melt(id_vars=['order_id','category'],
```

```
value_vars=['revenue','profit','cogs'],
var_name='metric', value_name='amount')
# explode: expand list-valued cells
# (useful if cells contain lists after str.split)
```

14.4 Export

```
# To CSV
df.to_csv('output.csv', index=False)
# To Excel (requires openpyxl)
df.to_excel('output.xlsx', sheet_name='Sales', index=False)
# To JSON
df.to_json('output.json', orient='records', indent=2)
# To clipboard (for quick pasting into Excel)
df.to_clipboard(index=False)
```

14.5 Questions & Answers

Q1:

Build a full method-chaining pipeline: filter Completed orders, add margin %, rank by revenue, keep top 100.

Answer:

Use query(), assign(), and nlargest() in a chain.

```
top100 = (
df
.query('status == "Completed"')
.assign(margin = lambda d: (d['profit']/d['revenue']*100).round(2))
.nlargest(100, 'revenue')
.reset_index(drop=True)
)
print(top100[['order_id','product_name','revenue','margin']].head(10))
```

Q2:

Melt revenue, profit, and cogs into a long format DataFrame.

Answer:

melt() turns selected value columns into rows.

```
long = df.melt(
id_vars=['order_id','category','order_date'],
value_vars=['revenue','profit','cogs'],
var_name='metric', value_name='amount')
print(long.head(9))
```

Q3:

Downcast all numeric columns to save memory and report the saving.

Answer:

Use pd.to_numeric with downcast, then compare memory_usage.

```
before = df.memory_usage(deep=True).sum()
num_cols = df.select_dtypes('number').columns
for c in num_cols:
if df[c].dtype == 'int64':
```

```
    df[c] = pd.to_numeric(df[c], downcast='integer')
else:
    df[c] = pd.to_numeric(df[c], downcast='float')
after = df.memory_usage(deep=True).sum()
print(f'Saved {(before-after)/1024:.1f} KB')
```

Q4:

Export the top 50 revenue orders to a CSV file.

Answer:

nlargest then to_csv.

```
df.nlargest(50,'revenue').to_csv('top50_orders.csv', index=False)
print('Exported!')
```

CHAPTER APP

Quick Reference Cheat Sheet

Essential Methods at a Glance

Category	Method / Property	Purpose
Loading	pd.read_csv(f, parse_dates=[c])	Load CSV with date parsing
Inspection	df.info(), df.describe()	Types, stats summary
Inspection	df.shape, df.dtypes	Dimensions and types
Selection	df[['c1','c2']]	Multi-column selection
Selection	df.loc[mask, cols]	Label-based filter+select
Selection	df.iloc[r, c]	Position-based select
Filtering	df[cond1 & cond2]	Boolean AND filter
Filtering	df.query('expr')	SQL-style filter
Filtering	df['c'].isin(list)	Membership test
Missing	df.isnull().sum()	Count NaN per column
Missing	df.fillna(val)	Fill NaN values
Missing	df.dropna(subset=[c])	Drop rows with NaN in col
Strings	df['c'].str.contains(pat)	Regex/substring check
Strings	df['c'].str.extract(r'(pat)')	Capture group extraction
Transform	df['c'].apply(func)	Apply function element-wise
Transform	df.assign(new=lambda d: ...)	Chain new columns
Transform	pd.cut(s, bins, labels)	Custom binning
Transform	pd.get_dummies(df['c'])	One-hot encoding
Groupby	df.groupby('c').agg(...)	Split-apply-combine
Groupby	df.groupby('c').transform('sum')	Group stats same shape
Groupby	df.groupby('c').filter(func)	Keep groups by condition
Pivot	pd.pivot_table(df,...)	Excel-style pivot
Pivot	pd.crosstab(a, b)	Frequency cross table
Pivot	.stack() / .unstack()	Reshape multi-index
Merge	pd.merge(a, b, on=c, how='left')	SQL-style join
Merge	pd.concat([a,b], ignore_index=T)	Stack DataFrames
Time	pd.to_datetime(s)	Parse date strings
Time	s.dt.year / .month / .day_name()	Date component access
Time	ts.resample('ME').sum()	Time-period aggregation

Window	s.rolling(7).mean()	Moving average
Window	s.expanding().sum()	Cumulative sum
Window	s.cumsum() / .cummax()	Running cumulative
Sort/Rank	df.sort_values('c', asc=False)	Sort DataFrame
Sort/Rank	s.rank(method='dense')	Rank Series
Sort/Rank	df.nlargest(n,'c')	Top n rows
Stats	s.mean() / .median() / .std()	Descriptive stats
Stats	s.corr(s2)	Pearson correlation
Stats	s.quantile(.90)	Percentile
Advanced	df.pipe(func)	Method chain helper
Advanced	df.melt(id_vars, value_vars)	Wide to long reshape
Advanced	df.memory_usage(deep=True)	Memory inspection
Export	df.to_csv('f.csv', index=False)	Save to CSV
Export	df.to_excel('f.xlsx')	Save to Excel

Dataset Column Reference

Column	Type	Description
order_id	str	Unique order ID (ORD-XXXX)
order_date	datetime	Order date (parse with to_datetime)
month	str	YYYY-MM formatted month
quarter	str	Q1-2023 etc.
product_id	str	Product code P001-P015
product_name	str	Full product name
category	str	Electronics / Furniture / Stationery
customer_id	str	Customer code C001-C015
customer_name	str	Full customer name
region	str	Bangladesh division
customer_type	str	Corporate / SMB / Individual
salesperson	str	Sales rep name
channel	str	Online / In-Store / Reseller / Direct
quantity	int	Units ordered
unit_price	float	Price per unit
discount_pct	int	Discount percentage (0/5/10/15/20)
discount_amount	float	Discount in currency
revenue	float	Net revenue after discount

cogs	float	Cost of goods sold
profit	float	Revenue minus COGS
payment_method	str	Cash / Card / Mobile Banking / Bank Transfer
status	str	Completed / Returned / Pending
rating	float	Customer rating 1-5 (~20% missing)
return_reason	str	Reason for return (empty if none)